

MINIPROYECTO

Desarrollo de Aplicaciones Web



EL LATIDO DEL LINCE

Plataforma Web para la Gestión de Voluntariado y Donaciones de una ONG de Conservación del Lince Ibérico

Documentación Técnica del Proyecto

Autor	Alejandro
Titulación	Desarrollo de Aplicaciones Web
Fecha	Junio de 2026
Versión	1.0 — Versión Final

PHP 8.2 (MVC Nativo) · MySQL 10.4 (MariaDB) · Tailwind CSS · HTML5 · JavaScript

Tabla de Contenidos

1. Introducción.....	2
1.1 Descripción General del Proyecto.....	2
1.2 Motivación y Justificación.....	2
1.3 Objetivos del Proyecto.....	2
2. Análisis de Requisitos.....	3
2.1 Requisitos Funcionales.....	3
2.2 Requisitos No Funcionales.....	3
3. Arquitectura del Sistema.....	4
3.1 Patrón de Diseño MVC.....	4
3.2 Estructura de Directorios.....	4
3.3 Flujo de Petición HTTP.....	4
4. Diseño de la Base de Datos.....	5
4.1 Descripción General.....	5
4.2 Descripción de Tablas.....	5
4.3 Diagrama Entidad-Relación.....	6
5. Implementación Técnica.....	7
5.1 Capa de Conexión a la Base de Datos.....	7
5.2 Módulo de Autenticación y Seguridad.....	7
5.3 Módulo de Gestión de Quedadas.....	8
5.4 Módulo de Donaciones.....	8
5.5 Panel de Administración.....	9
6. Tecnologías y Herramientas Utilizadas.....	10
7. Seguridad de la Aplicación.....	11
8. Plan de Pruebas.....	12
9. Guía de Instalación y Despliegue.....	13
10. Conclusiones.....	14

1. Introducción

1.1 Descripción General del Proyecto

El presente documento recoge la documentación técnica de un miniproyecto desarrollado en el marco del módulo de Desarrollo de Aplicaciones Web (DAW). El proyecto consiste en el desarrollo completo de una plataforma web orientada a la gestión interna de una Organización No Gubernamental (ONG) dedicada a la conservación del Lince Ibérico (*Lynx pardinus*), especie catalogada como vulnerable en España y en proceso de recuperación gracias a programas de cría en cautividad y reintroducción.

La plataforma cubre dos necesidades fundamentales de este tipo de organizaciones: la coordinación de actividades de voluntariado medioambiental, denominadas internamente «quedadas», y la gestión de un sistema de donaciones económicas que permite a ciudadanos y simpatizantes contribuir de forma monetaria a los objetivos de conservación. El sistema implementa un ciclo completo de usuario, desde el registro y autenticación hasta la participación activa en actividades y el seguimiento de sus aportaciones.

1.2 Motivación y Justificación

La elección de esta temática responde a la convergencia de dos intereses complementarios: la voluntad de desarrollar un sistema web de complejidad real y la sensibilidad hacia la problemática de la biodiversidad en la Península Ibérica. El Lince Ibérico ha sido históricamente el felino más amenazado del planeta, y en la actualidad su recuperación depende en gran medida de la implicación de voluntarios y donantes que apoyan las labores de conservación de los hábitats.

Desde el punto de vista técnico, el proyecto permite poner en práctica los conocimientos adquiridos durante el curso en un entorno de complejidad práctica: se aplica el patrón de diseño MVC de forma estricta, se implementan mecanismos de autenticación y autorización por roles, se diseña y normaliza una base de datos relacional con restricciones de integridad referencial, y se construyen interfaces de usuario modernas y responsivas mediante Tailwind CSS.

1.3 Objetivos del Proyecto

1.3.1 Objetivos Funcionales

- Desarrollar un sistema de registro e inicio de sesión de usuarios con asignación de roles diferenciados (administrador y usuario estándar).
- Implementar un módulo de gestión de eventos de voluntariado («quedadas») con control automático de aforo.
- Desarrollar un sistema de donaciones que soporte tanto aportaciones identificadas como anónimas.
- Construir un panel de administración completo para la gestión de usuarios, eventos y estadísticas.

1.3.2 Objetivos Técnicos

- Implementar el patrón arquitectónico MVC de forma nativa en PHP 8.2, sin dependencia de frameworks externos.
- Garantizar la seguridad del sistema mediante sesiones nativas de PHP, hashing de contraseñas con `password_hash()` y sentencias preparadas (PreparedStatements) con MySQLi.
- Diseñar una base de datos relacional en MariaDB 10.4 con integridad referencial completa mediante claves foráneas con borrado en cascada.
- Maquetar una interfaz de usuario completamente responsiva mediante el framework CSS utilitario Tailwind CSS.

2. Análisis de Requisitos

2.1 Requisitos Funcionales

La siguiente tabla recoge los requisitos funcionales del sistema, clasificados por módulo y priorizados según su criticidad para el funcionamiento de la plataforma.

ID	Módulo	Descripción del Requisito	Prioridad
RF-01	Autenticación	El sistema permitirá el registro de nuevos usuarios con nombre de usuario, correo electrónico y contraseña.	Alta
RF-02	Autenticación	Las contraseñas se almacenarán cifradas mediante el algoritmo bcrypt (password_hash con PASSWORD_DEFAULT).	Alta
RF-03	Autenticación	El sistema validará las credenciales en el proceso de login y establecerá una sesión PHP segura con datos del usuario y su rol.	Alta
RF-04	Autenticación	El sistema bloqueará el acceso tras 3 intentos fallidos de inicio de sesión consecutivos.	Alta
RF-05	Autenticación	El usuario podrá activar la opción «Recordarme» para persistir su nombre de usuario mediante una cookie con TTL de 30 días.	Media
RF-06	Usuarios	Cada usuario dispondrá de un perfil editable donde podrá modificar su nombre de usuario, contraseña y foto de perfil.	Media
RF-07	Usuarios	El sistema validará el formato y la extensión de las imágenes de perfil subidas, limitando el tamaño máximo a 2 MB.	Media
RF-08	Quedadas	Los usuarios autenticados podrán visualizar el listado de eventos de voluntariado con estado 'disponible'.	Alta
RF-09	Quedadas	Los usuarios podrán inscribirse a una quedada con un clic, siempre que existan plazas libres.	Alta
RF-10	Quedadas	El sistema incrementará automáticamente el contador de plazas ocupadas al confirmar una inscripción.	Alta
RF-11	Quedadas	El sistema impedirá la inscripción duplicada de un mismo usuario en la misma quedada.	Alta
RF-12	Quedadas	Los usuarios podrán darse de baja de una quedada, decrementando el contador de plazas.	Alta
RF-13	Donaciones	Los usuarios (identificados o anónimos) podrán realizar donaciones indicando nombre, importe y método de pago.	Alta
RF-14	Donaciones	El sistema calculará dinámicamente el total acumulado de donaciones respecto al objetivo global.	Alta
RF-15	Admin	El rol 'admin' tendrá acceso exclusivo a un panel de control con estadísticas y gestión de entidades.	Alta
RF-16	Admin	El administrador podrá crear, editar y archivar quedadas (borrado lógico mediante cambio de estado).	Alta
RF-17	Admin	El administrador podrá gestionar usuarios: editar sus datos y eliminarlos del sistema.	Alta

2.2 Requisitos No Funcionales

ID	Categoría	Descripción
RNF-01	Seguridad	Toda entrada de usuario será saneada con htmlspecialchars() antes de ser procesada.

RNF-02	Seguridad	Todas las consultas a la base de datos se ejecutarán mediante PreparedStatements con MySQLi para prevenir inyecciones SQL.
RNF-03	Seguridad	Las rutas de acceso al panel de administración verificarán la sesión y el rol antes de servir cualquier contenido.
RNF-04	Usabilidad	La interfaz será completamente responsiva y funcional en dispositivos móviles, tablets y escritorio.
RNF-05	Rendimiento	Las consultas a la base de datos estarán optimizadas mediante índices en las claves primarias, foráneas y campos UNIQUE.
RNF-06	Mantenibilidad	El código fuente seguirá la arquitectura MVC, con separación estricta de responsabilidades entre Modelo, Vista y Controlador.
RNF-07	Portabilidad	La aplicación será desplegable en cualquier entorno LAMP/WAMP con PHP 8.2+ y MySQL/MariaDB 10.4+.

3. Arquitectura del Sistema

3.1 Patrón de Diseño MVC

La aplicación implementa de forma estricta el patrón arquitectónico Modelo-Vista-Controlador (MVC), uno de los patrones de diseño más extendidos en el desarrollo de aplicaciones web. Este patrón establece una separación clara de responsabilidades entre tres capas lógicas, lo que facilita el mantenimiento, la escalabilidad y la legibilidad del código.

A diferencia de frameworks que abstraen este patrón automáticamente (como Laravel o Symfony), en este proyecto el MVC se ha implementado de forma nativa y explícita en PHP 8.2, lo que demuestra una comprensión profunda del patrón y de sus mecanismos de comunicación.

Capa	Directorio	Responsabilidad
Modelo (M)	Modelo/	Contiene la lógica de acceso a datos. Cada archivo encapsula las funciones de consulta SQL para una entidad concreta. No contiene lógica de presentación ni de negocio.
Vista (V)	vista/	Contiene las plantillas HTML/Tailwind que se presentan al usuario. Las vistas reciben datos ya procesados del Controlador y los renderizan.
Controlador (C)	Controlador/	Actúa como intermediario entre la Vista y el Modelo. Gestiona la sesión, sanea y valida los datos de entrada, invoca al Modelo y redirige al destino correspondiente.
Conexión	conexion/	Centraliza la inicialización del recurso de conexión MySQLi. Este recurso se inyecta en los modelos mediante inclusión directa.
Componentes	Partes/	Almacena fragmentos HTML reutilizables (header y footer) que se incluyen mediante require/include para evitar duplicidad de código.

3.2 Estructura de Directorios

La siguiente estructura refleja la organización del código fuente del proyecto de forma fiel al repositorio entregado:

```

ONG/
├── conexion/
│   └── conexion_base_datos.php      # Inicialización de la conexión MySQLi
├── Controlador/
│   ├── controlador_admin.php        # Panel de administración (CRUD
│   │                               usuarios/quedadas)
│   ├── controlador_crear_quedada.php # Creación de nuevas quedadas
│   ├── controlador_donacion.php      # Procesamiento de donaciones
│   └── controlador_editar_quedada.php # Edición de quedadas existentes

```

```

├── controlador_editar_usuario.php# Edición del perfil de usuario
├── controlador_inscripcion.php  # Alta/baja en quedadas
├── controlador_login.php        # Autenticación y creación de sesión
├── controlador_logout.php       # Destrucción de sesión
├── controlador_perfil.php        # Vista del perfil de usuario
├── controlador_quedadas.php     # Listado de quedadas
├── controlador_recompensas.php  # Módulo de recompensas
├── Modelo/
│   ├── modelo_admin.php        # Estadísticas y operaciones de
administración
│   ├── modelo_donaciones.php   # CRUD y agregados de donaciones
│   ├── modelo_quedadas.php     # CRUD e inscripciones de quedadas
│   └── modelo_usuarios.php     # Registro, login, validación y perfil
├── Partes/
│   ├── header.php              # Cabecera HTML reutilizable (nav,
sesión)
│   └── footer.php              # Pie de página HTML reutilizable
├── vista/
│   ├── admin_panel.php         # Panel de administración (vista)
│   ├── admin_editar_usuario.php # Formulario de edición de usuario
(admin)
│   ├── crear_quedada.php       # Formulario de nueva quedada
│   ├── donaciones.php          # Página de donaciones
│   ├── editar_quedada.php      # Formulario de edición de quedada
│   ├── editar_usuario.php      # Formulario de edición de perfil
│   ├── lince.php              # Página informativa sobre el lince
│   ├── login.php              # Formulario de inicio de sesión
│   ├── perfil_usuario.php      # Panel de perfil del usuario
│   ├── quedadas.php           # Listado de eventos de voluntariado
│   ├── recompensas.php        # Página de recompensas por donación
│   ├── registro.php           # Formulario de registro de usuarios
└── index.php                   # Página principal / Landing page

```

3.3 Flujo de Petición HTTP

El ciclo de vida de una petición típica en el sistema sigue la siguiente secuencia:

1. El usuario interactúa con una Vista (formulario, botón) y genera una petición HTTP (GET o POST).
2. La petición llega al Controlador correspondiente, que inicia o verifica la sesión PHP.
3. El Controlador sanea los datos de entrada con `htmlspecialchars()` y los valida mediante funciones del Modelo.
4. Si la validación es correcta, el Controlador invoca las funciones del Modelo que ejecutan PreparedStatements contra la base de datos.
5. El Modelo devuelve el resultado al Controlador.
6. El Controlador almacena mensajes de feedback en variables de sesión y redirige mediante `header('Location: ...')` a la Vista apropiada.
7. La Vista recupera los datos de sesión y los renderiza al usuario.

4. Diseño de la Base de Datos

4.1 Descripción General

La base de datos, denominada bdong, se encuentra alojada en un servidor MariaDB 10.4 (compatible con MySQL 8.x). Consta de cinco tablas relacionales que modelan las entidades principales del dominio: usuarios, quedadas (eventos de voluntariado), inscripciones (relación N:M entre usuarios y quedadas), donaciones y objetivos. El motor de almacenamiento empleado es InnoDB en todas las tablas, lo que garantiza soporte para transacciones ACID y para las restricciones de integridad referencial mediante Foreign Keys.

El conjunto de caracteres empleado es utf8mb4 con collation utf8mb4_general_ci, lo que garantiza la correcta representación de caracteres del alfabeto español (tildes, ñ) y de emojis unicode.

4.2 Descripción de Tablas

4.2.1 Tabla: usuarios

Almacena las credenciales y los datos de perfil de todos los usuarios registrados en la plataforma. Aplica restricciones UNIQUE en los campos usuario y email para garantizar la unicidad de los identificadores de acceso.

Campo	Tipo	Restricciones	Descripción
id	INT(11)	PK, AUTO_INCREMENT, NOT NULL	Identificador único del usuario.
usuario	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de usuario para el login. Único en el sistema.
email	VARCHAR(100)	NOT NULL, UNIQUE	Correo electrónico del usuario. Único en el sistema.
contrasena	VARCHAR(255)	NOT NULL	Hash bcrypt de la contraseña (output de password_hash).
rol	VARCHAR(20)	NOT NULL, DEFAULT 'usuario'	Rol de acceso: 'usuario' o 'admin'.
fecha_registro	TIMESTAMP	NOT NULL, DEFAULT current_timestamp()	Fecha y hora de creación de la cuenta.
foto_perfil	VARCHAR(255)	DEFAULT 'avatar_default.jpg'	Nombre del archivo de imagen de perfil.

4.2.2 Tabla: quedadas

Almacena los eventos de voluntariado organizados por la ONG. El campo estado implementa un mecanismo de borrado lógico que permite «archivar» eventos sin eliminarlos físicamente de la base de datos, preservando así la integridad histórica de las inscripciones.

Campo	Tipo	Restricciones	Descripción
id	INT(11)	PK, AUTO_INCREMENT, NOT NULL	Identificador único del evento.
titulo	VARCHAR(100)	NOT NULL	Nombre descriptivo del evento de voluntariado.
descripcion	TEXT	NOT NULL	Descripción detallada de la actividad a realizar.
fecha	DATE	NOT NULL	Fecha de celebración del evento.
hora_inicio	TIME	NOT NULL	Hora de inicio de la actividad.

hora_fin	TIME	NOT NULL	Hora de finalización prevista.
ubicacion	VARCHAR(255)	NOT NULL	Nombre del lugar o paraje natural donde se celebra.
provincia	VARCHAR(50)	NOT NULL	Provincia donde se ubica el evento.
plazas_totales	INT(11)	NOT NULL	Número máximo de voluntarios permitidos.
plazas_ocupadas	INT(11)	DEFAULT 0	Contador de inscripciones activas. Se actualiza automáticamente.
estado	VARCHAR(50)	NOT NULL, DEFAULT 'disponible'	Estado del evento: 'disponible' o 'archivada'.

4.2.3 Tabla: inscripciones

Tabla de asociación que modela la relación muchos a muchos (N:M) entre usuarios y quedadas. La clave compuesta (id_usuario, id_quedada) garantiza la unicidad de la inscripción, evitando duplicados a nivel de base de datos. Ambas claves foráneas están configuradas con ON DELETE CASCADE, lo que asegura la limpieza automática de inscripciones huérfanas cuando se elimina un usuario o se elimina una quedada.

Campo	Tipo	Restricciones	Descripción
id	INT(11)	PK, AUTO_INCREMENT, NOT NULL	Identificador único de la inscripción.
id_usuario	INT(11)	NOT NULL, FK → usuarios(id) ON DELETE CASCADE	Referencia al usuario inscrito.
id_quedada	INT(11)	NOT NULL, FK → quedadas(id) ON DELETE CASCADE	Referencia al evento al que se inscribe.
fecha_inscripcion	TIMESTAMP	NOT NULL, DEFAULT current_timestamp()	Momento exacto de la inscripción.

4.2.4 Tabla: donaciones

Registra cada transacción económica realizada en la plataforma. Admite tanto donaciones de usuarios registrados (id_usuario referencia a la tabla usuarios) como donaciones anónimas (id_usuario es NULL). En el caso de donantes identificados, la clave foránea también tiene ON DELETE CASCADE, garantizando la eliminación de las donaciones asociadas cuando se borra un usuario.

Campo	Tipo	Restricciones	Descripción
id	INT(11)	PK, AUTO_INCREMENT, NOT NULL	Identificador único de la donación.
id_usuario	INT(11)	FK → usuarios(id) ON DELETE CASCADE, NULLABLE	Referencia al usuario registrado. NULL si es donación anónima.
nombre_donante	VARCHAR(100)	NULLABLE	Nombre del donante (libre, para donaciones anónimas).
email_donante	VARCHAR(150)	NULLABLE	Email del donante (solo para donaciones anónimas).
cantidad	DECIMAL(10,2)	NOT NULL	Importe económico de la donación en euros.
fecha_donacion	TIMESTAMP	NOT NULL, DEFAULT current_timestamp()	Fecha y hora del registro de la donación.
metodo_pago	VARCHAR(50)	NULLABLE	Método de pago utilizado (ej. 'tarjeta').

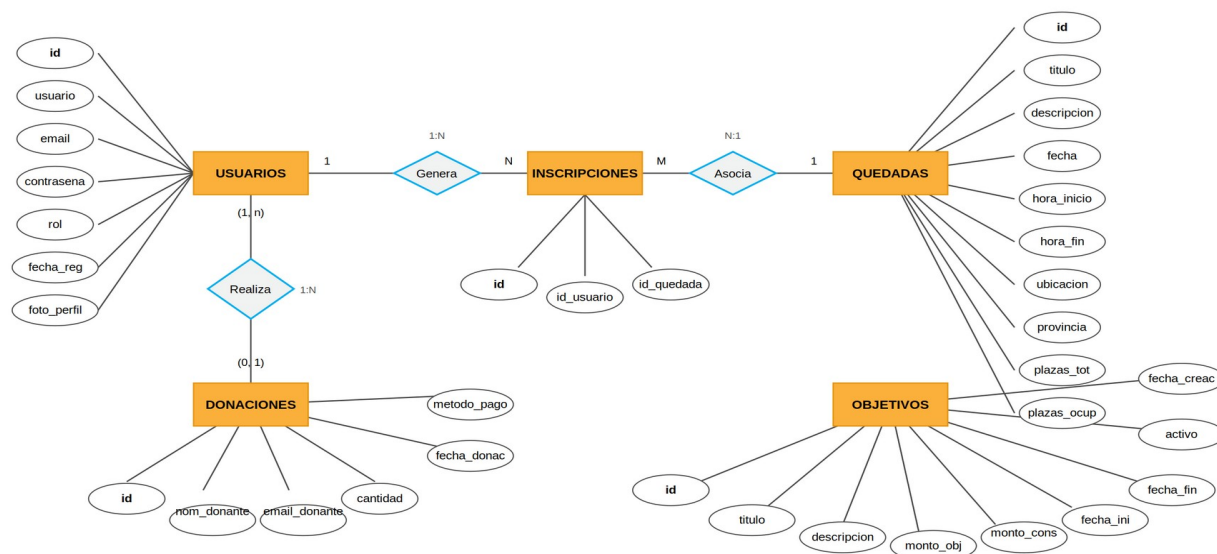
4.2.5 Tabla: objetivos

Almacena las campañas de recaudación activas de la ONG. Cada objetivo define un importe meta (monto_objetivo), lleva el seguimiento del importe acumulado (monto_conseguido), tiene un período de vigencia (fecha_inicio / fecha_fin) y un flag booleano activo que permite al sistema identificar el objetivo en curso para calcular el porcentaje de progreso mostrado en la vista de donaciones.

Campo	Tipo	Restricciones	Descripción
id	INT(11)	PK, AUTO_INCREMENT, NOT NULL	Identificador único del objetivo.
titulo	VARCHAR(150)	NOT NULL	Nombre descriptivo de la campaña de recaudación.
descripcion	TEXT	NULLABLE	Descripción detallada del destino de los fondos.
monto_objetivo	DECIMAL(10, 2)	NOT NULL	Importe económico total que se desea alcanzar.
monto_conseguido	DECIMAL(10, 2)	DEFAULT 0.00	Importe acumulado hasta la fecha. Se actualiza con cada donación.
fecha_inicio	DATE	NULLABLE	Fecha de inicio de la campaña.
fecha_fin	DATE	NULLABLE	Fecha de cierre prevista de la campaña.
activo	TINYINT(1)	NOT NULL, DEFAULT 1	Flag booleano. 1 = objetivo activo y visible en la plataforma.
fecha_creacion	TIMESTAMP	NOT NULL, DEFAULT current_timestamp()	Fecha de creación del registro en el sistema.

4.3 Diagrama Entidad-Relación

El siguiente diagrama muestra las entidades principales del sistema, sus atributos y las relaciones entre ellas, siguiendo la notación Chen clásica:



- **USUARIOS** — genera → **DONACIONES**: Relación 1:N opcional. Un usuario puede realizar cero o más donaciones. Las donaciones anónimas no tienen usuario asociado (clave foránea nullable).
- **USUARIOS** — se inscribe ↔ **QUEDADAS**: Relación N:M resuelta mediante la tabla **INSCRIPCIONES**. Un usuario puede inscribirse a múltiples quedadas, y una quedada puede tener múltiples usuarios inscritos. La tabla **INSCRIPCIONES** actúa como tabla de asociación con atributo propio (fecha_inscripcion).
- **OBJETIVOS**: Entidad independiente que define las campañas de recaudación. No tiene relación directa con otras tablas mediante FK; el sistema calcula el progreso consultando el SUM de la tabla donaciones y comparándolo con el monto_objetivo del objetivo activo.

5. Implementación Técnica

5.1 Capa de Conexión a la Base de Datos

La conexión a la base de datos se gestiona en el archivo `conexion/conexion_base_datos.php`, que inicializa un recurso de conexión MySQLi nativo. Este archivo es incluido mediante `require_once` en todos los controladores que necesitan interactuar con la base de datos, garantizando que existe una única instancia de la conexión por ciclo de petición.

```
<?php
$conexion = mysqli_connect('localhost', 'root', '', 'bdong');
if (!$conexion) {
    die("Error de conexion: " . mysqli_connect_error());
}
?>
```

Se ha optado por MySQLi en lugar de PDO por su integración directa con MySQL/MariaDB y por la facilidad que ofrece para el manejo de PreparedStatements procedurales, que es el estilo empleado de forma consistente a lo largo de todo el proyecto.

5.2 Módulo de Autenticación y Seguridad

5.2.1 Registro de Usuarios

El proceso de registro implementa tres niveles de validación antes de persistir el registro en la base de datos:

8. Validación de formato en el Modelo: se verifica que el nombre de usuario tenga entre 3 y 20 caracteres, que solo contenga caracteres alfanuméricos y guiones bajos (regex `/^[a-zA-Z0-9_]+$`), y que el email tenga formato válido mediante `filter_var()` con el filtro `FILTER_VALIDATE_EMAIL`.
9. Validación de unicidad en base de datos: antes de la inserción, se ejecuta una consulta `PreparedStatement` que comprueba si el nombre de usuario o el email ya existen en el sistema.
10. Cifrado de contraseña: la contraseña se cifra con `password_hash($password, PASSWORD_DEFAULT)`, que aplica el algoritmo `bcrypt` con un factor de coste adaptativo.

5.2.2 Inicio de Sesión

El controlador de login implementa las siguientes medidas de seguridad:

- Saneamiento previo de los datos del formulario mediante `htmlspecialchars()`.
- Consulta a la base de datos por nombre de usuario mediante `PreparedStatement`, recuperando el hash almacenado.
- Verificación del hash mediante `password_verify()`, que compara la contraseña en texto plano con el hash `bcrypt` almacenado.
- Contador de intentos fallidos almacenado en sesión. Tras 3 intentos fallidos, el sistema redirige a una vista de bloqueo y deniega futuros intentos.
- Establecimiento de sesión con los campos: `$_SESSION['usuario']`, `$_SESSION['id']`, `$_SESSION['rol']` y `$_SESSION['foto_perfil']`.
- Cookie de «Recordarme» con TTL de 30 días, establecida únicamente si el usuario marca la casilla correspondiente.

5.2.3 Control de Acceso por Roles

Todos los controladores que sirven funcionalidades restringidas inician el bloque de ejecución con una verificación de sesión y rol. El siguiente fragmento, presente en `controlador_admin.php`, ilustra este mecanismo:

```
session_start();
if (!isset($_SESSION['usuario']) || $_SESSION['rol'] !== 'admin') {
    header("Location: ../vista/login.php");
    exit();
}
```

De forma análoga, los controladores de funcionalidades de usuario (inscripción a quedadas, edición de perfil) verifican que existe una sesión activa antes de procesar cualquier dato, redirigiendo al login en caso contrario. La llamada a `exit()` tras el header de redirección es obligatoria para detener la ejecución del script y prevenir el acceso a código posterior.

5.3 Módulo de Gestión de Quedadas

5.3.1 Visualización y Filtrado

El modelo `modelo_quedadas.php` implementa la función `obtener_resultado_quedadas()`, que recupera únicamente los eventos con estado 'disponible', ordenados por fecha ascendente. Esto garantiza que las quedadas archivadas no sean visibles para los usuarios estándar.

```
function obtener_resultado_quedadas($conexion) {
    $sql = "SELECT id, titulo, descripcion, fecha, hora_inicio, hora_fin,
```

```
        ubicacion, provincia, plazas_totales, plazas_ocupadas, estado
    FROM quedadas WHERE estado = 'disponible' ORDER BY fecha ASC";
$stmt = mysqli_prepare($conexion, $sql);
if ($stmt) {
    mysqli_stmt_execute($stmt);
    return mysqli_stmt_get_result($stmt);
}
return false;
}
```

5.3.2 Sistema de Inscripciones

El proceso de inscripción se implementa mediante el controlador `controlador_inscripcion.php`, que gestiona tanto la alta como la baja de un usuario en una quedada. El sistema actualiza el campo `plazas_ocupadas` de la tabla `quedadas` de forma atómica, utilizando operaciones de incremento/decremento SQL (`plazas_ocupadas + 1`) en lugar de lecturas previas, lo que minimiza el riesgo de condiciones de carrera en entornos concurrentes.

La detección de inscripciones duplicadas se delega en la restricción `UNIQUE KEY` de la base de datos (definida sobre la clave compuesta `id_usuario, id_quedada`). El modelo captura la excepción MySQLi con código 1062 (`duplicate entry`) y devuelve el valor de cadena `'duplicado'` al controlador, que muestra el mensaje de feedback apropiado.

5.3.3 Borrado Lógico (Archivado)

El administrador no puede eliminar físicamente una quedada con inscripciones activas sin perder la coherencia histórica. Por este motivo, se implementa un mecanismo de borrado lógico: en lugar de un `DELETE`, el controlador ejecuta un `UPDATE` que cambia el campo `estado` de `'disponible'` a `'archivada'`. Las quedadas archivadas dejan de ser visibles en el listado de usuarios pero permanecen en la base de datos, preservando el historial de participación.

5.4 Módulo de Donaciones

El módulo de donaciones está diseñado para soportar dos tipos de aportantes:

- **Usuarios identificados:** si existe una sesión activa, el `id` del usuario se recupera de `$_SESSION['id']` y se asocia a la donación. El campo `id_usuario` de la tabla `donaciones` apunta al registro del usuario.
- **Donantes anónimos:** si no existe sesión, el campo `id_usuario` se establece como `NULL` y el donante proporciona libremente su nombre y correo electrónico en el formulario.

El modelo implementa la función `obtener_total_donaciones_totales()`, que ejecuta un `SELECT SUM(cantidad)` para calcular el acumulado global en tiempo real. Este valor se compara con el objetivo de recaudación definido para calcular el porcentaje de progreso que se muestra en la barra de progreso de la vista.

La redirección post-donación apunta a la vista `recompensas.php`, donde se muestra al donante un mensaje de agradecimiento personalizado junto con información sobre el impacto de su aportación.

5.5 Panel de Administración

El panel de administración (`controlador_admin.php` + vista `admin_panel.php`) consolida en una única interfaz las operaciones de gestión del sistema. Implementa un mecanismo de pestañas (tabs) persistentes mediante una variable de sesión (`$_SESSION['activeTab']`), lo que permite al administrador navegar entre las secciones de «Quedadas» y «Usuarios» sin perder el contexto de la operación que estaba realizando.

Las estadísticas del panel se obtienen mediante la función `obtener_estadisticas_admin()` del modelo `modelo_admin.php`, que ejecuta tres consultas agregadas: `COUNT` de quedadas, `COUNT` de usuarios y `SUM` de plazas_ocupadas.

Funcionalidad	Controlador	Modelo involucrado
Ver listado de quedadas (admin)	<code>controlador_admin.php</code>	<code>modelo_quedadas.php</code> → <code>obtener_quedadas_admin()</code>
Crear nueva quedada	<code>controlador_crear_quedada.php</code>	<code>modelo_quedadas.php</code> → <code>insertar_quedada()</code>
Editar quedada existente	<code>controlador_editar_quedada.php</code>	<code>modelo_quedadas.php</code> → <code>actualizar_quedada()</code>
Archivar quedada	<code>controlador_admin.php</code>	<code>modelo_quedadas.php</code> → <code>cambiar_estado_quedada()</code>
Ver listado de usuarios (admin)	<code>controlador_admin.php</code>	<code>modelo_usuarios.php</code> → <code>obtener_todos_usuarios()</code>
Editar datos de usuario	<code>controlador_admin.php</code>	<code>modelo_usuarios.php</code> → <code>actualizar_usuario_admin()</code>
Eliminar usuario	<code>controlador_admin.php</code>	<code>modelo_usuarios.php</code> → <code>eliminar_usuario()</code>
Ver estadísticas globales	<code>controlador_admin.php</code>	<code>modelo_admin.php</code> → <code>obtener_estadisticas_admin()</code>

6. Tecnologías y Herramientas Utilizadas

6.1 Stack Tecnológico

Tecnología	Versión	Rol en el Proyecto
PHP	8.2.12	Lenguaje de servidor. Implementa los Controladores y Modelos del patrón MVC. Se utiliza en su variante procedimental con MySQLi nativo.
MariaDB / MySQL	10.4.32	Sistema gestor de bases de datos relacionales. Almacena la totalidad de la información persistente de la aplicación.
Tailwind CSS	3.x (CDN)	Framework CSS utilitario. Gestiona la maquetación, el diseño responsivo y los componentes visuales de todas las vistas.
HTML5	—	Lenguaje de marcado para la estructura semántica de todas las vistas de la aplicación.
JavaScript (Vanilla)	ES6+	Utilizado para interacciones de cliente: validaciones de formulario en tiempo real, efectos visuales y comportamiento dinámico de la UI.
MySQLi (extensión PHP)	—	API de acceso a base de datos. Se utilizan PreparedStatements para prevenir inyecciones SQL.
Apache HTTP Server	—	Servidor web local empleado durante el desarrollo (entorno XAMPP / WAMP).

6.2 Herramientas de Desarrollo

Herramienta	Propósito
Visual Studio Code	Editor de código fuente principal con extensiones para PHP, Tailwind CSS y MySQL.
XAMPP / phpMyAdmin	Entorno de desarrollo local (Apache + PHP + MariaDB) y cliente web para gestión de la base de datos.
Git	Control de versiones del proyecto. El repositorio incluye historial de commits desde

	la fase inicial de desarrollo.
GitHub	Plataforma de alojamiento del repositorio remoto y respaldo del código fuente.
Figma / Diseño propio	Diseño previo de wireframes y definición de la paleta de colores de la interfaz.

7. Seguridad de la Aplicación

7.1 Resumen de Medidas de Seguridad Implementadas

La seguridad de una aplicación web es un requisito transversal que afecta a todas las capas de la arquitectura. En este proyecto se han implementado medidas de mitigación para los principales vectores de ataque contemplados en la guía OWASP Top 10.

Amenaza	Medida Implementada	Capa
SQL Injection (A03)	Uso exclusivo de PreparedStatements con MySQLi (mysqli_prepare + bind_param) en todas las consultas que reciben datos del usuario.	Modelo
XSS (A03 - Stored/Reflected)	Saneamiento de todas las entradas con htmlspecialchars() antes de su procesamiento en el Controlador.	Controlador
Broken Authentication (A07)	Hashing de contraseñas con bcrypt (password_hash + password_verify). Contraseñas nunca almacenadas en texto plano.	Controlador / BD
Broken Access Control (A01)	Verificación de sesión y rol al inicio de todos los controladores de funcionalidades restringidas. Uso de exit() tras las redirecciones.	Controlador
Credential Stuffing	Bloqueo de acceso tras 3 intentos fallidos de login, gestionado mediante contador en sesión.	Controlador
Path Traversal (subida de archivos)	Validación de extensión de archivos de imagen (whitelist: jpg, jpeg, png, gif) y limitación de tamaño a 2 MB.	Modelo
Session Fixation	Gestión correcta de sesiones con session_unset() + session_destroy() en el proceso de logout.	Controlador

8. Plan de Pruebas

8.1 Pruebas Funcionales

Las pruebas funcionales verifican que cada requisito funcional identificado en el apartado 2 se satisface correctamente. Se han ejecutado de forma manual, simulando el comportamiento de distintos roles de usuario.

ID Prueba	Caso de Prueba	Resultado Esperado	Resultado
PT-01	Registro con datos válidos	El usuario queda registrado y se redirige al login.	OK
PT-02	Registro con usuario duplicado	Se muestra el error 'El nombre de usuario ya está en uso'.	OK
PT-03	Login con credenciales correctas (rol usuario)	Se inicia sesión y se redirige al perfil del usuario.	OK
PT-04	Login con credenciales correctas (rol admin)	Se inicia sesión y se redirige al controlador_admin.	OK
PT-05	Login con contraseña incorrecta	Se muestra el error 'Usuario o	OK

		contraseña incorrectos'.	
PT-06	Bloqueo por 3 intentos fallidos	Se redirige a la vista de bloqueo y se deniega el acceso.	OK
PT-07	Inscripción a quedada con plazas libres	Se registra la inscripción y se incrementa plazas_ocupadas.	OK
PT-08	Inscripción duplicada a la misma quedada	Se muestra el error '¡Ya estás inscrito en esta quedada!'.	OK
PT-09	Darse de baja de una quedada	Se elimina la inscripción y se decrementa plazas_ocupadas.	OK
PT-10	Acceso al panel admin sin sesión de admin	Redirección automática al login.	OK
PT-11	Donación identificada (usuario con sesión)	Se registra la donación con id_usuario correcto.	OK
PT-12	Donación anónima (sin sesión)	Se registra la donación con id_usuario NULL.	OK
PT-13	Archivar quedada desde panel admin	La quedada cambia a estado 'archivada' y desaparece del listado de usuarios.	OK
PT-14	Crear nueva quedada desde panel admin	La quedada se inserta y aparece en el listado de admin.	OK

8.2 Pruebas de Seguridad

ID Prueba	Escenario de Ataque	Resultado Esperado	Resultado
PS-01	Intento de SQL Injection en campo usuario del login	La consulta PreparedStatement parametriza el valor; no se altera la lógica SQL.	OK
PS-02	Intento de XSS en campo de texto del formulario de registro	El valor es escapado por htmlspecialchars(); no se ejecuta código JavaScript en la vista.	OK
PS-03	Acceso directo por URL a /Controlador/controlador_admin.php sin sesión	Redirección inmediata a /vista/login.php.	OK
PS-04	Subida de archivo PHP con extensión renombrada a .jpg	El validador de extensión rechaza el archivo.	OK

9. Guía de Instalación y Despliegue

9.1 Requisitos del Sistema

Componente	Versión Mínima
PHP	8.0 o superior (recomendado 8.2)
MySQL / MariaDB	10.3 o superior
Servidor Web	Apache 2.4+ o Nginx con soporte para PHP-FPM
Extensiones PHP	mysqli, mbstring, fileinfo, session

9.2 Pasos de Instalación en Entorno Local (XAMPP)

11. Clonar o descomprimir el repositorio del proyecto en la carpeta htdocs de XAMPP:

```
# Ejemplo de ruta en Windows
```

```
C:\xampp\htdocs\ONG\
```

12. Iniciar los servicios de Apache y MySQL desde el panel de control de XAMPP.
13. Acceder a phpMyAdmin (<http://localhost/phpmyadmin>) y crear una base de datos llamada bdong.
14. Importar el archivo de volcado SQL bdong.sql proporcionado en el repositorio desde la opción «Importar» de phpMyAdmin.
15. Verificar que el archivo conexion/conexion_base_datos.php apunta a los parámetros correctos del servidor local:

```
$conexion = mysqli_connect('localhost', 'root', '', 'bdong');
```

16. Acceder a la aplicación desde el navegador:

```
http://localhost/ONG/index.php
```

9.3 Credenciales de Prueba

Usuario	Contraseña	Rol
alejandro	(hash bcrypt en BD)	Administrador
pepito	(hash bcrypt en BD)	Usuario estándar

Las contraseñas se encuentran almacenadas como hashes bcrypt en el volcado SQL. Para conocer la contraseña en texto plano de los usuarios de prueba, consultar con el autor del proyecto.

10. Conclusiones

El desarrollo de este proyecto ha permitido consolidar y aplicar en un entorno de complejidad real los conocimientos técnicos adquiridos durante el Desarrollo de Aplicaciones Web. Se ha demostrado la capacidad de diseñar e implementar una arquitectura MVC completa sin dependencia de frameworks externos, lo que exige una comprensión profunda del patrón y de los mecanismos de comunicación entre capas.

Desde el punto de vista de la seguridad, se han aplicado de forma consistente las medidas de mitigación más relevantes del ecosistema web: prevención de inyecciones SQL mediante PreparedStatements, protección frente a XSS mediante saneamiento de entradas, control de acceso basado en roles mediante sesiones nativas de PHP, y cifrado de credenciales mediante bcrypt.

El resultado es una aplicación web funcional, con una interfaz de usuario moderna y responsiva, que resuelve de forma efectiva las necesidades de gestión de una ONG de carácter medioambiental. La temática escogida —la conservación del Lince Ibérico— ha dotado al proyecto de coherencia y de significado más allá del mero ejercicio técnico.

Fin del documento · El Latido del Lince — ONG · Alejandro · Junio 2026